Hospital Emergency – Room Triage System

Team No - 06

U. Bruhathi – 2620030025

M. Harshitha Reddy – 2620030118

1 - Immediate

2 - Emergent

3 - Urgent

4 - Less Urgent

5 - Non-Urgent

Abstract

Hospital Emergency Room Triage System — Abstract
The Hospital Emergency Room Triage System is a software application designed to streamline and automate the initial patient assessment, prioritization, and assignment process in a hospital emergency department. Traditional paper-based triage methods often lead to delays, human error, and difficulty in tracking patient urgency, which can compromise critical care delivery. This system provides a centralized platform to efficiently classify patients based on medical severity and manage emergency room workflows.
The system enables medical staff to register incoming patients, record vital signs, assign triage priority levels (e.g., Immediate, Urgent, Non-Urgent), and track waiting times. It maintains real-time records of patient status, bed availability, and attending staff assignments. Healthcare providers can search for patient records, update medical statuses, and identify critical cases requiring immediate intervention.
By automating routine intake and prioritizing care based on medical necessity, the proposed system reduces admission delays, minimizes manual record errors, saves crucial time, and improves patient outcome tracking. It provides an organized, reliable method for monitoring emergency room capacity and generating operational reports. The Hospital Emergency Room Triage System can be implemented as a Java application and further enhanced with features like real-time vital monitor

Understanding Problem Statement

THE PROBLEM

1. Emergency rooms receive more patients than can be treated at once.
2. Manual triage depends on judgement, so decisions can be slow and inconsistent.
3. Critical patients risk waiting behind less serious cases.

OBJECTIVE

Build a rule-based Java program that ranks arriving patients by urgency so the most critical are seen first.

OUR SOLUTION

1. Capture
   Nurse enters age and vital signs at th
2. Score
   Rules turn each reading into severity
3. Classify
   Total score maps to triage level 1 to
4. Prioritise
   Queue is ordered so critical cases cor

Module 1 (CO 1): Java Foundation

Module 1 Topics	Where it appears in the triage system
Algorithm & Flowchart	Triage steps planned as a flowchart before code is written
Structure of a Java program	Class TriageSystem with public static void main(String[] args)
Data Types, variables, constants	int age, double temperature, Boolean such as 38.6, 'Y' and true
Identifiers & Literals	heartRate, patientName; literals such as 38.6, 'Y' and true
Operators	Arithmetic, relational & logical, ternary, and bitwise (alert flags)
Typecasting & expressions	(double) totalWait / count for average wait; precedence in score formulas
Console I/O	Scanner reads the vitals; printf prints an aligned triage report

Module 1 (CO 1) : Flowchart & Code



Variables, constants & operators

```
final int MAX_PATIENTS = 50;
int heartRate = 118, spo2 = 92;
double temp = 38.6;
int score = 0;
score += (heartRate > 110) ? 2 : 0;
if (temp >= 38 && spo2 < 94)
score++;
double avg = (double) totalWait /
count;
```

Console I/O with Scanner & printf

```
Scanner sc = new Scanner(System.in);
System.out.print("Patient name: ");
String name = sc.nextLine();
System.out.print("Heart rate (bpm): ");
int heartRate = sc.nextInt();
System.out.printf("%-10s Score:%2d
Level:%d%n", name, score, level);
```

Module 2 (CO 2): Selection

If-else-if ladder, nested if, switch

```
if (score >= 9) level = 1;
else if (score >= 6) level = 2;
else if (score >= 4) level = 3;
else if (score >= 2) level = 4;
else level = 5;
// nested if: safety override
if (spo2 < 90) {
  if (!conscious) level = 1;
}
switch (level) {
  case 1: label = "IMMEDIATE";
break;
  case 2: label = "EMERGENT"; break;
  default: label = "STANDARD";
}
```

Five triage levels

1. Level 1: Immediate
   Score 9 or more | seen at once
2. Level 2: Emergent
   Score 6 to 8 | target 10 min
3. Level 3: Urgent
   Score 4 to 5 | target 30 min
4. Level 4: Less Urgent
   Score 2 to 3 | target 60 min
5. Level 5: Non-Urgent
   Score 0 to 1 | target 120 min

Module 2 (CO 2): Iteration

While loop

Main menu repeats until the nurse chooses Exit.

```
while (choice != 0) {
  showMenu();
}
```

Do-while loop

Ask for heart rate at least once; repeat until valid.

```
do {
  hr = sc.nextInt();
} while (hr < 20 || hr > 250);
```

For loop

Traverse the waiting list and print each patient.

```
for (int i = 0; i < n; i++) {
  print(queue[i]);
}
```

Continue

Skip patients who are already discharged.

```
for (int i = 0; i < n; i++) {
  if (done[i]) continue;
  print(queue[i]);
}
```

Module 3 (CO 3): Methods

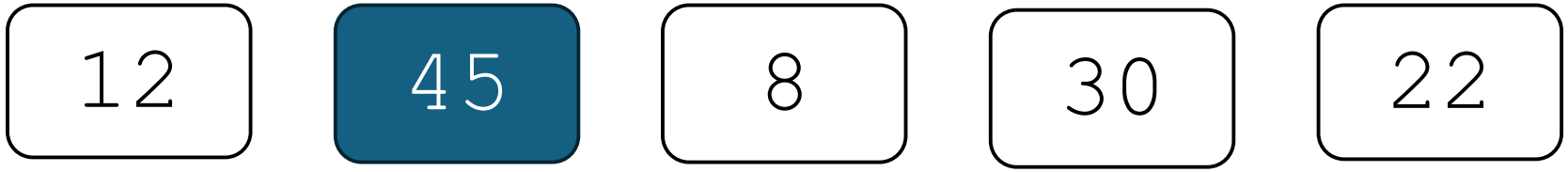
Methods

```
static int calcScore(int hr, int spo2, double temp) {
  int score = 0;
  // ... rules add points ...
  return score;
}
int score = calcScore(118, 92, 38.6);
// overloading: same name, diff params
static void print(String name, int level) { }
static void print(String name, int level, int wait) { }
```

Each method does one job. Overloading lets print() serve both quick and de

Module 3 (CO 3): 1D Arrays

```
int[ ] waitTimes = {12, 45, 8, 30,
22}; // minutes
```



Index starts at 0, so length is 5 and the last index is 4.

117
Sum (min)

23.4
Average (min)

45
Longest wait

1
Waits over 30 min

Module 3 (CO 3): 2D Arrays & Matrices

Day 1 Arrivals

	L1	L2	L3	L4	L5
Morning	1	3	6	8	4
Evening	2	4	7	9	5
Night	1	2	5	4	2

Day 2 Arrivals

	L1	L2	L3	L4	L5
Morning	0	4	5	7	6
Evening	1	5	8	6	3
Night	2	1	4	5	1

Total = Day1 + Day2

	L1	L2	L3	L4	L5
Morning	1	7	11	15	10
Evening	3	9	15	15	8
Night	3	3	9	9	3

Module 4 (CO 4) : Classes & Objects

Class, fields & constructors

```
Patient is a blueprint; each
arrival is an object.
class Patient {
  private String name; private int age, level;
  Patient(String name, int age) {
    this.name = name; this.age = age; }
  Patient(String name) { this(name, 0); }
}
Patient p = new Patient("Asha", 34);
```

Encapsulation

```
Private fields; access only
through methods
public int getLevel() { return level; }
public void setLevel(int level) {
  if (level >= 1 && level <= 5)
    this.level = level;
}
// invalid levels are rejected
```

Static members & constants

```
Shared by the whole class, not one
patient.
static int totalPatients = 0;
static final int MAX_LEVEL = 5;
static int getTotal() {
  return totalPatients;
}
// constructor: totalPatients++;
```

toString() & null references

```
Readable output for reports and
the queue.
@Override
public String toString() {
  return name + " (L" + level + ")";
}
Patient next = null;
if (next != null) print(next);
```

Module 4 (CO 4): Inheritance & Exceptions

Inheritance & overriding

```
CriticalPatient "is-a" Patient with extra rules
class CriticalPatient extends Patient {
  @Override
  public String toString() {
    return "URGENT: " + super.toString();
  }
}
```

Interface

```
Programming to an interface, not a class.
interface Triageable {
  int calcScore();
}
class Patient implements Triageable { }
Triageable t = new CriticalPatient(..);
```

Try-catch-finally & multi-catch

```
Bad input must not crash the ER system.
try { hr = Integer.parseInt(s); }
catch (NumberFormatException |
       NullPointerException ex) {
  System.out.println("Invalid input");
} finally { System.out.println("Done"); }
```

Throw & throws

```
Checked vs unchecked exceptions
static void check(int hr)
    throws InvalidVitalsException {
  if (hr < 20 || hr > 250)
    throw new InvalidVitalsException("HR");
}
// unchecked: ArithmeticException
```

Module 5 (CO 5): Strings

Core String operations

```
Strings are immutable; a new
String is returned.
```

```
String n = "  asha rao ".trim();
n.length();  n.charAt(0);
n.substring(0, 4);
n.toUpperCase();
n.contains("rao");  n.indexOf('r');
```

StringBuilder

```
Mutable text for building the
triage report.
```

```
StringBuilder sb = new StringBuilder();
sb.append(name).append(" L").append(level);
sb.append('\n');
sb.insert(0, "> ");
String report = sb.toString();
```

Equals() vs ==

```
Compare values, never references.
```

```
String a = "Asha";
String b = new String("Asha");
a == b;       // false (reference)
a.equals(b);  // true  (value)
a.equalsIgnoreCase("ASHA");  // true
```

Split(), join() & validation

```
Parse a record line into fields
```

```
String line = "Asha,34,118,92,38.6";
String[] f = line.split(",");
int age = Integer.parseInt(f[1].trim());
if (f.length != 5) { /* malformed */ }
String out = String.join(" | ", f);
```

Module 5 (CO 5): Sorting & File I/O

Comparable & Comparator

```
Order the queue: level first, then
wait time
class Patient implements Comparable<Patient> {
  public int compareTo(Patient o) {
    return Integer.compare(level, o.level);
  }
}
Comparator<Patient> byWait =
  Comparator.comparingInt(Patient::getWait);
```

Sort & binary search

```
   Library algorithms, no hand-written
                              sort.
Arrays.sort(waitTimes);
Collections.sort(list);
Collections.sort(list, byWait);
int pos = Arrays.binarySearch(
    waitTimes, 30);  // array must be sorted
```

Read & write files

```
Save patients using java.nio.file.
Path p = Path.of("patients.csv");
List<String> lines = Files.readAllLines(p);
Files.writeString(p, rec + "\n",
  StandardOpenOption.APPEND,
  StandardOpenOption.CREATE);
```

Try-with-resources & bad records

```
Missing file or bad line never stops the
run
try (BufferedReader br =
     Files.newBufferedReader(p)) {
  String ln;
  while ((ln = br.readLine()) != null) {
    if (ln.split(",").length != 5) continue;
  }
} catch (IOException ex) { /* no file */ }
```

Module 6 (CO 6): Collections & Generics

ArrayList<Patient>

Typed list: the compiler blocks wrong types.

```
List<Patient> queue = new ArrayList<>();
queue.add(new Patient("Asha", 34));
queue.remove(0);
queue.get(0);  queue.size();
for (Patient p : queue) print(p);
```

HashMap: patient lookup

Find a patient by ID in one step.

```
Map<Integer, Patient> byId = new HashMap<>();
byId.put(101, p);
Patient f = byId.get(101);
byId.containsKey(101);
for (var en : byId.entrySet()) { }
```

HashSet & ordered TreeMap

No duplicates; keys kept sorted.

```
Set<String> allergies = new HashSet<>();
allergies.add("Penicillin");  // no repeats
Map<Integer, Integer> perLevel =
    new TreeMap<>();
perLevel.merge(level, 1, Integer::sum);
```

Autoboxing & iterators

int and Integer convert automatically.

```
List<Integer> waits = new ArrayList<>();
waits.add(45);       // autoboxing
int w = waits.get(0); // unboxing
Iterator<Patient> it = queue.iterator();
while (it.hasNext()) {
 if (it.next().done) it.remove(); }
```

Module 6 (CO 6): Lambdas & Streams

Lambdas & functional interfaces

Pass behaviour as a parameter.

```
Predicate<Patient> critical = p -> p.level == 1;
Function<Patient, String> nm =
Patient::getName;
Consumer<Patient> show = p -> print(p);
Supplier<Patient> blank = () -> new
Patient("?");
```

Stream pipeline

Declarative processing of the queue.

```
queue.stream()
  .filter(critical)
  .sorted(byWait)
  .limit(3)
  .map(Patient::getName)
forEach(System.out::println);
```

Terminal operations

Summaries in a single line.

```
long n = queue.stream().filter(critical).count();
int total = waits.stream()
  .reduce(0, Integer::sum);
List<String> names = queue.stream()
  .map(Patient::getName).distinct().toList();
```

Collectors & file streams

Build shift reports from data.

```
Map<Integer, Long> perLevel =
queue.stream()
  .collect(Collectors.groupingBy(
     Patient::getLevel, Collectors.counting()));
String s =
queue.stream().map(Patient::getName)
  .collect(Collectors.joining(", "));
Files.lines(p).forEach(System.out::println);
```

Geo Tag Photo and git hub link



https://github.com/bruhathi0920

https://github.com/maddhikaharshithar